
From DDPM to Rectified Flow: a Measured, From-Scratch Diffusion Study

Yi Hou

University of Chinese Academy of Sciences
Beijing, China
houyi25@mailsucas.ac.cn

Abstract

This report documents a from-scratch implementation and measurement study of the modern diffusion stack. No `diffusers`, `flash_attn` or pretrained generative component is used: DDPM, DDIM, a diffusion transformer (DiT) with adaLN-Zero conditioning, and rectified flow are all implemented in this repository, together with the training pipeline, the sampler suite and the evaluation code. The study is organized as a controlled 2×2 design (UNet vs. DiT $\times$ ϵ -prediction vs. rectified flow) at two scales, MNIST 28×28 and CelebA 64×64 , with every cell trained under one frozen protocol. Headline results: DDIM at 50 function evaluations matches the 1000-step ancestral sampler on CelebA (FID 11.45 vs. 11.37) at $20 \times$ the sampling throughput, and rectified flow dominates the quality–NFE frontier on MNIST (FID 0.88 at 64 evaluations, versus 4.32 for the best ϵ -prediction sampler at 50). The same deterministic sampler that improves the UNet collapses on the transformer: its FID grows from 49 to 672 as NFE goes $10 \rightarrow 50$, against 33 for 1000-step ancestral sampling of the same weights. An η sweep shows the failure is the missing noise injection. On the systems side, bf16 buys $3.15 \times$ step time and $1.6 \times$ memory over fp32 while `torch.compile` buys nothing ($1.00 \times$), and a kernel-level profile attributes 67% of GPU time to matmuls and 16% to elementwise work, which is where a fused adaLN kernel would have to earn its keep.

1 Introduction

Diffusion models are usually consumed through a library: a scheduler object, a pretrained backbone, a pipeline. This report takes the opposite route. Everything is built from scratch, including the forward and reverse processes, the two backbones, both training objectives, the samplers and the evaluation.

The study asks three questions:

- **RQ1 (architecture).** At a similar parameter scale, does replacing a UNet with a transformer (DiT) backbone help or hurt generation quality and computational efficiency?
- **RQ2 (objective and sampler).** How does a rectified-flow objective change the quality–NFE trade-off, and how much does sampler choice alone buy for a fixed trained model (DDPM ancestral vs. DDIM)?
- **RQ3 (systems).** How much training and inference throughput can be recovered by mixed precision, compilation and kernel-level fusion at this scale — and where does a distributed strategy stop being worth it?

Section 2 puts the three objectives on a common footing. Section 3 describes the implementation, Section 4 the frozen protocol and the evaluation protocol, Sections 5–7 the results, and Section 8 collects the experiments that failed, which were as instructive as the ones that worked.

Table 1: Results at a glance. All numbers are measured on the hardware described in Section 4; the raw archives live in `results/*.json`.

Question	Baseline	Measured effect
RQ1 backbone	UNet (11.7M, 2.43 GFLOPs/sample)	DiT (16.5M, 1.08): worse FID at every matched budget (MNIST ϵ : 2.79 vs. 33.2; flow: 0.88 vs. 4.99) despite $2.25\times$ fewer FLOPs
RQ2 objective	ϵ -prediction	rectified flow: FID 0.88 vs. 4.32 at NFE 64 vs. 50; crossover near NFE 16
RQ2 sampler	DDPM ancestral (1000 NFE)	DDIM at 50 NFE: FID 11.45 vs. 11.37 on CelebA at $20\times$ the throughput; collapses on the DiT ($49 \rightarrow 672$ as NFE $10 \rightarrow 50$)
RQ3 systems	FP32, uncompiled	BF16 $3.15\times$ step time, peak memory $32.9 \rightarrow 21.1$ GB, MFU $10.4\% \rightarrow 32.7\%$; compile $1.00\times$
RQ3 profile	—	matmul 67.6%, elementwise 15.3%, attention 11.1%; the fused-adaLN target is the element-wise slice

Table 2: The three objectives on a common footing. $\bar{\alpha}_t$ is the usual DDPM cumulative product; the flow variant uses a continuous $t \in [0, 1]$.

Method	Path x_t	Target	Sampler (NFE)
DDPM [1]	$\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$	ϵ	ancestral, stochastic (1000)
DDIM [2]	same	ϵ	deterministic ODE (10–50)
Rectified flow [8, 9]	$(1 - t) x_0 + t \epsilon$	$v = \epsilon - x_0$	Euler ODE (4–128)

2 Background: one path, three objectives

All three methods place the data and the noise at the ends of an interpolation path and train a network to regress a target along it. They differ in the path, in the target, and in the solver used to walk back from noise to data.

Both backbones embed the timestep with a sinusoidal basis calibrated for $t \in [0, 1000]$, the DDPM range. The flow code keeps its path mathematics in $t \in [0, 1]$ and passes $t \times 1000$ to the network; without that scaling the embedding is nearly constant and conditioning collapses.

3 Implementation

Everything is written in PyTorch, with no generative-model library. Table 3 lists the measured sizes. The parameter ranking and the FLOPs ranking disagree across scales: on CelebA the UNet is larger on both axes, while on MNIST it has fewer parameters but more FLOPs.

Backbones. The UNet is a residual U-net with GroupNorm, SiLU, sinusoidal timestep injection into every residual block, and `scaled_dot_product_attention` at the bottleneck. The DiT follows (author?) [7]: patchify with a strided convolution, fixed two-dimensional sincos positional embeddings, and blocks whose attention and MLP are modulated by an adaLN-Zero MLP that is zero-initialized, so every block starts as the identity (verified in `tests/test_sanity.py`: the untrained model outputs exactly zero). Attention uses PyTorch SDPA in both backbones; no fused-attention library is used.

Training. A single training loop drives both objectives. It supports `torchrun` DDP, YAML configs, checkpoint resume, an exponential moving average of the weights (all reported FID uses EMA weights), per-epoch EMA-only snapshots for quality-vs-progress curves, and cost accounting: parameters, measured FLOPs, step time, throughput, peak memory and MFU are written to `results/<run>.json` on every run.

Samplers. Ancestral DDPM sampling, deterministic DDIM sampling with a configurable NFE, and Euler integration of the flow ODE. All three reuse the same trained ϵ - or velocity-predicting network, so they can be compared at fixed weights.

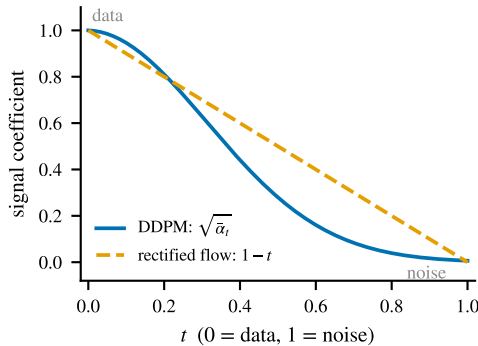


Figure 1: The diffusion path and the rectified-flow path between data and noise. The flow objective regresses a constant velocity along a straight line; the diffusion objective traverses a curved, non-uniform-speed path.

Table 3: Measured sizes (forward pass, one sample at the model’s native resolution). Params and FLOPs are counted by `eval/cost.py` with `torch.utils.flop_counter`, so both backbones are measured with the same tool.

Dataset	Backbone	Params	GFLOPs/sample	Conditioning	Attention
MNIST 28 ²	UNet	11.7M	2.43	sinusoidal + MLP	bottleneck
MNIST 28 ²	DiT	16.5M	1.05	adaLN-Zero	all 6 blocks (49 tokens)
CelebA 64 ²	UNet	174.1M	67.8	sinusoidal + MLP	bottleneck
CelebA 64 ²	DiT	129.6M	43.6	adaLN-Zero	all 12 blocks (256 tokens)

4 Experimental setup

Hardware. One node with six NVIDIA RTX A6000 GPUs (48 GB each) connected over PCIe with *no* NVLink. The GPUs span two NUMA domains — 0–3 on one socket, 4–5 on the other — and the cross-domain link traverses PCIe *and* the inter-socket interconnect. This topology is why every multi-GPU measurement in this report runs inside a single domain and sets `NCCL_P2P_DISABLE=1` (Section 8).

The node is shared. During these runs four other training jobs held between 0.5 and 47 GB on the remaining cards, so the GPU was never exclusively ours. The single-GPU settings sweep (30 steps plus five discarded warmups, coefficient of variation 0.1–0.6%) is the least exposed to that contention; the throughput columns of the FID tables are global rates from 4-GPU sharded sweeps and should be read as such. Software: Python 3.11 and PyTorch 2.13.0 (+cu126).

Protocol. All MNIST cells share one frozen protocol: identical optimizer, learning-rate schedule, batch size (global), EMA decay and seed (Table 4). Unlike recipes that tune each cell separately, this design means a difference between cells is attributable to the backbone or the objective rather than to the optimizer.

FID protocol. Reference statistics are computed from the training tensor itself (`data/celeba_64_uint8.pt` for CelebA, the MNIST training split for MNIST) with the same preprocessing chain used to evaluate samples; third-party statistics are never used, because any difference in resizing makes the numbers incomparable. CelebA uses the standard Inception-v3 pool3 features at 299². Inception features are not meaningful for grayscale digits, so the MNIST cells are evaluated in the feature space of a small convolutional classifier trained on MNIST inside this repository (Table 4 footnote); that metric is *non-standard* and only comparable within this report. Tier 1 evaluations use 10k samples, Tier 2 (final) 50k, and the spread is reported as the variability across five random 10k subsets.

Table 4: The frozen protocol. Any deviation is recorded in the run’s JSON.

	MNIST 28 ²	CelebA 64 ²
Cells	UNet/DiT $\times \epsilon$ /flow, one run each	
Epochs	200	300
Global batch	256	800 (160 per GPU $\times$ 5)
Optimizer	Adam, lr 2×10^{-4} , warmup then cosine to 10^{-6}	
Gradient clipping	1.0	
EMA decay	0.9999, applied in every cell	
Seed	42	

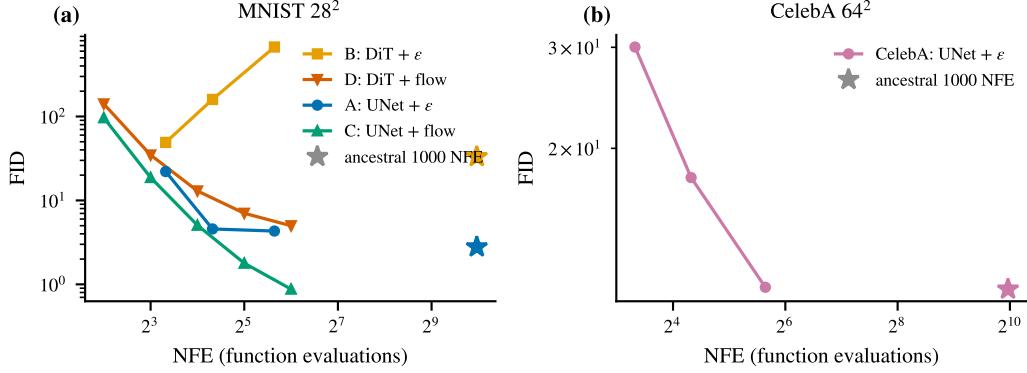


Figure 2: Quality against the number of function evaluations, by dataset (note the logarithmic axes and the separate panels: the CelebA FIDs sit an order of magnitude above the MNIST ones). Within a panel every curve is a trained model plus a sampler; the DDPM vs. DDIM pair is a same-weights sampler ablation, while comparisons across cells also change the training objective.

5 Sampler study (RQ2)

5.1 What a better sampler buys, at CelebA scale

The practical promise of DDIM is that a fixed trained model can be sampled in far fewer network evaluations. Table 5 answers that for the existing CelebA DDPM checkpoint (UNet, 174.1M parameters, 300 epochs, trained before this protocol freeze). Quality improves monotonically with the budget, and the 50-step deterministic sampler lands within 1% of the 1000-step ancestral sampler while running 20 $\times$ faster: the last 950 evaluations of the ancestral chain buy 0.1 FID points and cost 7.1 hours per 50k-sample evaluation. The 10-step sampler is the other end of the trade — 5 $\times$ fewer evaluations than the 50-step sampler for 2.6 $\times$ the FID.

A 10k subset reads about 1 FID unit *higher* than the 50k measurement of the same sampler (12.41 $\pm$ 0.08 vs. 11.45 for DDIM at 50 NFE), so the 10k ancestral number in the last row is if anything pessimistic: on equal footing the gap between the two rows is smaller than the table’s raw numbers suggest. This sample-count dependence is why the deterministic rows here are reported at 50k and the expensive ancestral row is not.

5.2 Quality–NFE frontier on MNIST

Table 6 is the same measurement for all four MNIST cells, which share a training budget and protocol. The rectified-flow cells dominate ϵ -prediction once the budget exceeds $\sim$ 16 evaluations: at 64 evaluations the flow cells reach FID 0.88 (UNet) and 4.99 (DiT), where the best ϵ -prediction result is 4.32 at 50. Every curve is monotone in the budget except the transformer’s ϵ cell, which is the subject of the next subsection.

Table 5: CelebA 64²: one trained model, four sampling budgets (EMA weights). The deterministic rows are the 50k-sample tier; the ancestral row is the 10k tier because the 1000-step sampler costs 20 $\times$ more per sample (see the throughput column), and the protocol’s exception for it is recorded with the results. “Spread” is the FID across five random 10k subsets of the generated features — what the 10k protocol would have reported.

Sampler	NFE	n	FID	spread ($5 \times 10k$)	samples/s	50k evaluation
DDIM	10	50k	30.03	30.90 ± 0.15	194.2	4.3 min
DDIM	20	50k	17.79	18.72 ± 0.09	97.0	8.6 min
DDIM	50	50k	11.45	12.41 ± 0.08	38.8	21 min
ancestral	1000	10k	11.37	—	1.9	7.3 h

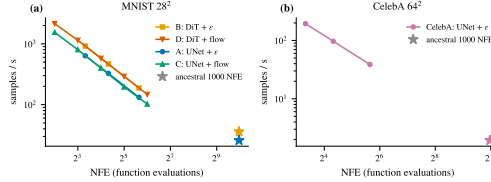


Figure 3: Sampling throughput against NFE, measured on the same runs. Throughput is inversely proportional to NFE (log–log slope ≈ -1 for every model), so the quality–NFE curves above are also quality–wall-clock curves up to a per-model constant.

5.3 The deterministic sampler collapses on the transformer

Cell B is the one place in this study where more computation makes the result worse: its FID *grows* from 49.1 at NFE 10 to 159.7 at 20 and 672.4 at 50, while 1000-step ancestral sampling of the same weights gives 33.2. Four checks localize the cause.

Not the weights. Sampling the raw (non-EMA) weights reproduces the failure (FID 273.5 and 274.6 at NFE 20 and 50), so the exponential moving average is not at fault.

Not the model. The identical checkpoint with the stochastic ancestral sampler reaches 33.2, and the training-time sample grids (ancestral, EMA weights) are recognizable digits. The transformer learned the distribution well enough to be sampled.

Not the sampler code. The same function improves the UNet monotonically ($22.06 \rightarrow 4.57 \rightarrow 4.32$); its arithmetic does not depend on the backbone.

It is the missing noise injection. DDIM’s η interpolates between the deterministic update ($\eta = 0$) and the ancestral posterior variance ($\eta = 1$). At fixed NFE = 50, raising η restores the data statistics monotonically (Table 7) and turns the samples from saturated blobs back into digits.

Why this happens. The deterministic recursion integrates the model’s ϵ error coherently along the trajectory, whereas each ancestral step re-injects the schedule’s noise and thereby bounds the drift. The transformer’s error is large enough for that difference to matter: measured against the same noised batches (`eval/eps_error_by_t.py`), its ϵ MSE is 30–55% higher than the UNet’s across the whole trajectory, with the gap widest at low noise levels — precisely where the deterministic update reads its final image directly out of the model’s x_0 estimate. The same sampler that is the cheapest way to sample the UNet is the worst way to sample the transformer, and the transformer is the model whose per-sample cost is lowest.

DDIM is not fragile in general: it is the best sampler in Table 5 and on the UNet. Its NFE reduction is conditional on score accuracy, and the condition is invisible unless both backbones are measured.

6 Backbone and objective (RQ1)

Table 8 is the headline comparison: four cells, one protocol, one dataset, and a FID for every one of them. The UNet wins both rows and both columns. Against the ϵ -objective it wins by $11.9\times$ (2.79 vs. 33.18); against rectified flow it wins by $5.7\times$ (0.88 vs. 4.99). The ordering holds at every sampling budget we measured (Table 6), so it is not an artefact of one sampler.

Table 6: MNIST FID (10k samples, EMA weights) as a function of the sampling budget. “—” marks budgets that were not run for that cell. The 1000 column is the ancestral DDPM sampler; the other columns are DDIM (for ϵ cells) or Euler (for flow cells).

Cell	Sampler	10	16	20	32	50	64	1000
A: UNet + ϵ	DDIM	22.06	—	4.57	—	4.32	—	2.79
B: DiT + ϵ	DDIM	49.14	—	159.68	—	672.39	—	33.18
C: UNet + flow	Euler	—	5.06	—	1.79	—	0.88	—
D: DiT + flow	Euler	—	12.98	—	7.03	—	4.99	—

Table 7: DiT + ϵ , DDIM at NFE = 50, as a function of the noise-injection level η . “Saturated” is the fraction of pixels pinned at ± 1 after the final clamp; MNIST digits are near-binary, so the ancestral sampler (NFE 1000, $\eta = 1$) saturates 84.8% of pixels. Nothing else about the sampler changes.

η	0	0.25	0.50	0.75	1.00
saturated pixels	47.7%	57.7%	66.0%	74.3%	79.1%
mean $ x $	0.904	0.908	0.916	0.927	0.934

Parameters are not compute. The DiT has *more* parameters (16.5M vs. 11.7M) and *less* compute (1.08 vs. 2.43 GFLOPs per sample): its parameters sit in a wide MLP over 49 tokens, while the UNet spends its compute on convolutions at 28^2 . The comparison is therefore not compute-matched, and the mismatch favours the winner — it would be a stronger claim if the two cells had equal FLOPs. This is the compute-matched experiment the report does not run.

The objective comparison is the cleaner of the two. Cells C and D differ from A and B only in the regression target and the sampler, and the same ordering appears at every budget: rectified flow reaches FID 0.88 where ϵ -prediction plateaus at 4.32. The flow objective buys an $8\times$ reduction in evaluations at the cheap end of the curve and a $4.9\times$ lower FID at the expensive end. A straighter path is easier to integrate, and the effect reproduces on both backbones.

7 Systems (RQ3)

Mixed precision is the whole story. BF16 buys $3.15\times$ the step rate and $1.6\times$ less peak memory, and nothing else we tried moves the needle. That single number is what makes the larger-scale experiments in this project affordable at all: the CelebA cells below cost ~ 36 and ~ 55 GPU-hours in fp32 and under 12 and 18 in bf16.

The speedup is not free. One MNIST cell was retrained under the identical protocol with the training step autocast to bf16, to price the speedup rather than assume it. The training loss is unchanged to three digits (0.0194 vs. 0.0193), but the sampled quality is not: the bf16 checkpoint scores FID 5.56 where the fp32 one scores 2.79 on the 1000-step ancestral sampler, while the two sit within noise of each other at 50 steps (4.10 vs. 4.32). This is the signature Section 5 found for the transformer, an error the long trajectory accumulates and the short one hides, appearing here from precision rather than from training. One cell and one seed is thin evidence, but it is enough that we do not quote $3.15\times$ as free, and it is why no quality number in this report comes from a mixed-precision run.

Compilation buys nothing here. `torch.compile` returns $1.00\times$, the same step time to three digits. Two explanations are consistent with the profile: 67% of GPU time is already inside cuBLAS matmuls that have little to fuse, and the fusible elementwise work sits between large kernels rather than accumulating in a long chain.

Why FSDP loses at this scale. FSDP is 12.8% slower than DDP and saves 4.8% of peak memory. The peak is dominated by activations: at batch 160 the model’s parameters, gradients and Adam state together account for roughly 2 GB of the 33.5 GB peak, so sharding them cannot help, while the all-gather and reduce-scatter on the critical path cost real time. Sharding pays when the model’s *states* rather than its activations set the ceiling, which at 130M parameters on 48 GB cards they do not.

Reading the MFU column. The 10.4% fp32 and 32.7% bf16 figures come from measured FLOPs (46.0 GFLOPs per sample, forward, counted by `torch.utils.flop_counter`) against the A6000’s

Table 8: The 2×2 design on MNIST, at the best budget measured for each cell (1000-step ancestral for the ϵ cells, 64-step Euler for the flow cells; both 10k samples, EMA weights). Sizes are measured, not configured, and by the same tool for both backbones.

Cell	Backbone + objective	Params	GFLOPs/sample	Best FID	NFE	s/step
A	UNet + ϵ	11.7M	2.43	2.79	1000	0.101
B	DiT + ϵ	16.5M	1.08	33.18	1000	0.073
C	UNet + flow	11.7M	2.43	0.88	64	0.102
D	DiT + flow	16.5M	1.08	4.99	64	0.072

Table 9: Training step cost for the CelebA DiT at 160 images per GPU under each execution setting (30 steps, 5 warmups discarded; single GPU unless noted). The last two rows are the distributed comparison at four GPUs. Throughput is global for the multi-GPU rows.

Setting	GPUs	s/step	img/s	peak GB	MFU	speedup
FP32	1	1.372	117	32.9	10.4%	1.00×
BF16	1	0.436	367	21.1	32.7%	3.15×
compile	1	1.377	116	32.9	10.4%	1.00×
BF16 + compile	1	0.439	365	21.1	32.5%	3.13×
FP32 + DDP	4	1.408	455	33.5	10.1%	—
FP32 + FSDP	4	1.588	403	31.9	9.0%	0.89× vs. DDP

dense peak. The counter covers matmul and convolution work only, so elementwise operations are invisible to it; a fused-adaLN kernel would have to earn its keep out of the 15.3% slice in Fig. 6, not out of the FLOPs number. The model also runs at 256 tokens with batch 160, which is not enough of either to saturate tensor cores, so bf16 moves this workload from barely using the tensor cores to using them moderately.

8 What didn’t work

Learning rate 5×10^{-4} collapses training. The first CelebA run used 5×10^{-4} on the assumption that Adam’s normalisation made it safe. The loss diverged and the samples degraded into noise, leaving no usable checkpoint: at that rate the early updates in the high-noise regime leave the region where the ϵ -prediction loss is informative, and the optimiser does not return. This protocol uses 2×10^{-4} with gradient clipping at 1.0 and logs the gradient norm every step, so the failure is visible before the loss is. Every cell in this report shares that pair of settings.

DataParallel deadlocks with a multi-worker DataLoader. The first multi-GPU configuration used `nn.DataParallel` and hung reliably, with no error output, whenever the DataLoader used worker processes. DataParallel replicates the module per forward pass and serialises it through one process, which interacts badly with forked workers pinning memory. The stack was rewritten around `torchrun`-native DDP, which every run here uses.

P2P hangs on this node’s NUMA layout. A hang remained after the move to DDP, and `nvidia-smi topo -m` explained it: the six GPUs split across two NUMA domains (0–3 and 4–5), with the cross-domain link traversing PCIe and the inter-socket interconnect. A job spanning both domains hung in its first collective; a job confined to one domain ran. NCCL’s peer-to-peer transport does not complete over that link, so every multi-GPU measurement here sets `NCCL_P2P_DISABLE=1` and stays inside one domain. On shared hardware the topology is part of the experiment setup, and measuring it once is cheaper than debugging it later.

FSDP is a net loss at this scale. Measured in Table 9 and accounted for in Section 7. It stays in the report because the criterion is the useful part: sharding pays when a model’s optimiser states, not its activations, set the memory ceiling.

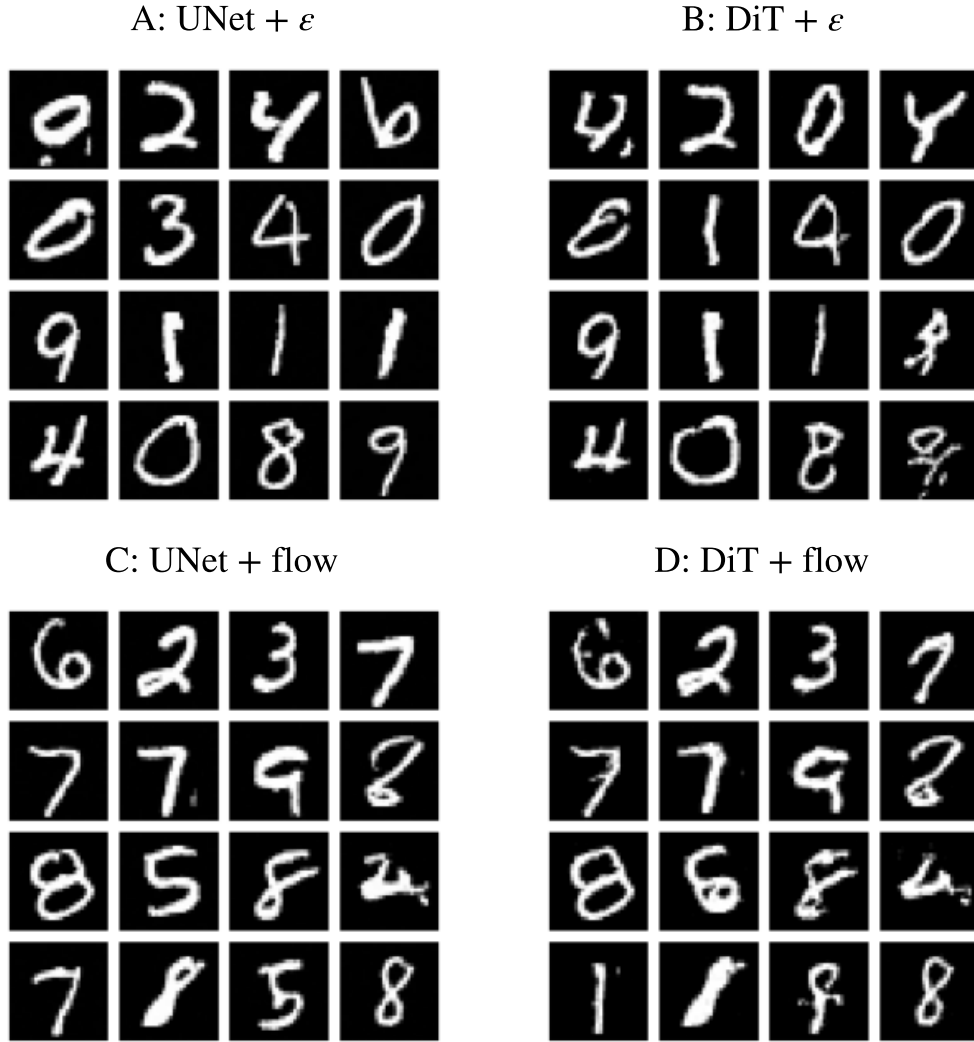


Figure 4: Final EMA samples, one block per cell (A: UNet + ϵ , B: DiT + ϵ , C: UNet + flow, D: DiT + flow). The transformer’s digits are thinner and more fragmented than the UNet’s, consistent with its FID.

9 Discussion and limitations

The three questions have different levels of support behind them. RQ2 is answered with a complete sweep at both scales. RQ3 is answered with measurements on one model and one hardware configuration, which is enough to establish the ranking of the levers (mixed precision $\gg$ everything else; compilation nothing; sharding negative) but not to generalise the constants. RQ1 has the weakest footing: the 2×2 is complete at MNIST scale, the comparison is not compute-matched, and there is no second scale — so the resulting ordering should be read as "at this size, under this protocol, with this much compute" rather than as a claim about transformers in general.

- Unconditional generation only: no classifier-free guidance, no text conditioning, no latent space.
- MNIST carries the full 2×2 ; CelebA contributes a sampler study on an existing checkpoint. The CelebA cells were estimated at ~ 36 h (DiT) and ~ 55 h (UNet) each in fp32 and were not run.

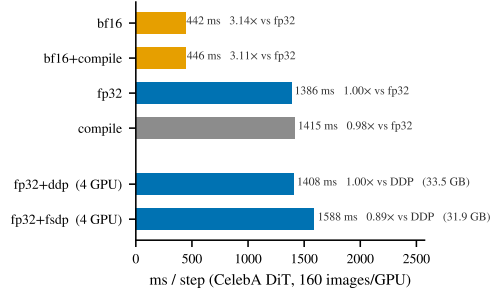


Figure 5: Step time by execution setting.

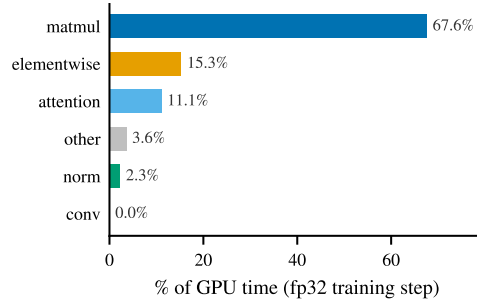


Figure 6: Where GPU time goes in a diffusion training step, by kernel category. This is a per-kernel accounting, which is precisely what a FLOPs count hides: the 15.3% spent on elementwise work does not appear in the 46.0 GFLOPs/sample figure at all.

- The A/B and C/D comparisons are not compute-matched: the UNet has $2.25\times$ the FLOPs per sample, so its win is achieved with more compute, not less.
- FID is computed against reference statistics built from this repository’s own preprocessing chain. Absolute values are *not* comparable with published numbers that use different resizing; only comparisons within this report are meaningful.
- The MNIST metric is a non-standard feature-space distance (a small CNN trained on MNIST for `eval/fid.py`), chosen because Inception features carry little information about grayscale digits.
- One seed per cell. The reported spread is FID variability across five random subsets, which is what the 10k-sample protocol would have given — not a confidence interval on the model.
- FLOPs counts cover matmul and convolution work only, so the MFU figures are a matmul-utilisation proxy rather than a hardware limit.
- The reported timings use fp32 training, and no quality number in this report comes from a mixed-precision run. bf16 was measured for speed (Table 9) and, in one cell, for quality: the training loss matched to three digits while the 1000-step FID doubled. That is one seed and one cell, so it locates a failure mode without marking where it starts.
- The transformer’s deterministic-sampler collapse (Section 5) is characterised at one scale with one model. The η sweep localises the mechanism to the absence of noise injection; whether the boundary moves with more training or more parameters is not measured.

Reproduction

All commands (environment, data, smoke tests, launching cells, evaluation) are in `reports/runbook.md`; the raw archive behind every number is `results/*.json`; the committed code contains the full training stack and the test suite used to validate the implementations (`tests/test_sanity.py`, 14 CPU-only invariants).

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- [2] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *ICLR*, 2021.
- [3] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *ICLR*, 2021.
- [4] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. *ICML*, 2021.
- [5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv:2207.12598*, 2022.
- [6] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *CVPR*, 2022.
- [7] William Peebles and Saining Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- [8] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: learning to generate and transfer data with rectified flow. *ICLR*, 2023.
- [9] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. *ICLR*, 2023.
- [10] Patrick Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. *ICML*, 2024.
- [11] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. *NeurIPS*, 2017.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [13] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: memory optimizations toward training trillion parameter models. *SC20*, 2020.
- [14] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. PyTorch FSDP: experiences on scaling fully sharded data parallel. *VLDB*, 2023.